

SECTION 1 GENERAL DESCRIPTION

The MC68881 and MC68882 floating-point coprocessors (FPCP) both fully implement the *IEEE Standard for Binary Floating-Point Arithmetic* (ANSI-IEEE Std 754-1985) for use with the Freescale M68000 Family of microprocessors. The coprocessors are both implemented in VLSI technology to give system designers the highest possible functionality in a physically small device. The MC68882 provides an increased level of performance in a coprocessor that is fully compatible and physically interchangeable with the MC68881.

Intended primarily for use as coprocessors to the MC68020/MC68030 32-bit microprocessor unit (MPU), the MC68881 and MC68882 provide a logical extension to the main processing unit integer data processing capabilities. These coprocessors provide a very high performance floating-point arithmetic unit and a set of floating-point data registers utilized in a manner that is analogous to the use of the integer data registers. The MC68881/MC68882 instruction set, a natural extension of all earlier members of the M68000 Family, supports all of the addressing modes of the host MPU. Due to the flexible bus interface of the M68000 Family, the MC68881 or MC68882 can be used with any of the MPU devices of the family and may also be used as a peripheral to other processors.

The major features of the MC68881 and MC68882 are:

- Eight general-purpose floating-point data registers, each supporting a full 80-bit extended precision real data format (a 64-bit mantissa plus a sign bit, and a 15-bit signed exponent).
- A 67-bit arithmetic unit to allow very fast calculations, with intermediate precision greater than the extended precision format.
- A 67-bit barrel shifter for high-speed shifting operations (for normalizing, etc.).
- Forty-six instructions, including 35 arithmetic operations.
- Full conformance to the ANSI-IEEE 754-1985 standard, including all requirements and suggestions.
- Support of functions not defined by the IEEE standard, including a full set of trigonometric and transcendental functions.
- Seven data formats: byte, word, and long word integers; single, double, and extended precision real numbers; and packed binary coded decimal string real numbers.
- Twenty-two constants available in the on-chip ROM, including π , e , and powers of 10.
- Virtual memory/machine operations.
- Efficient mechanisms for exception processing, context switches, and interrupt handling.
- Fully concurrent instruction execution with the main processor.
- Use with any host processor, on an 8-, 16-, or 32-bit data bus.



In addition to these features, the MC68882 provides:

- Concurrent execution of multiple floating-point instructions.
- Special-purpose hardware for high-speed conversion of binary real memory operands to/from the internal extended format.
- Simultaneous access to the floating-point registers by the MC68882's conversion and arithmetic processing units.
- Reduced coprocessor interface overhead to increase throughput.

1.1 THE COPROCESSOR CONCEPT

The FPCP functions as a coprocessor in systems where the MC68020 or MC68030 is the main processor via the M68000 coprocessor interface. It functions as a peripheral processor in systems where the main processor is the MC68000, MC68008, or MC68010.

The FPCP utilizes the M68000 Family coprocessor interface to provide a logical extension of the MPU registers and instruction set in a manner that is transparent to the programmer. The programmer perceives the MPU and FPCP execution model as if both devices were implemented on one chip.

A fundamental goal of the M68000 Family coprocessor interface is to provide the programmer with an execution model based upon sequential instruction execution by the MPU and the FPCP. For optimum performance, however, the coprocessor interface allows floating-point instructions to execute concurrently with MPU integer instructions. Concurrent instruction execution is further extended by the MC68882, which can execute multiple floating-point instructions simultaneously. However, the coprocessor interface and the FPCP are designed to maintain a strictly sequential instruction execution model from the programmer's viewpoint.

The FPCP is a non-DMA type coprocessor that uses a subset of the general-purpose coprocessor interface supported by the MPU. Features of the interface implemented in the FPCP are as follows:

- The main processor and the FPCP communicate via standard M68000 bus cycles.
- Communication between the main processor and the FPCP is not dependent upon the architecture of the individual devices (e.g., instruction pipes or caches, addressing modes).
- The main processor and the FPCP can operate at different clock speeds.
- The FPCP instructions support all addressing modes provided by the main processor.
- All effective addresses calculations and data transfers performed by the main processor at the request of the coprocessor.
- Overlapped (concurrent) instruction execution enhances throughput while maintaining the programmer's model of sequential instruction execution.
- Coprocessor detection of an exception that requires a trap to be taken is serviced by the main processor at the request of the FPCP.
- Support of virtual memory/virtual machine systems is provided via the FSAVE and FRESTORE instructions.

- Systems can use software emulation of the FPCP without reassembling or relinking user software.

1.2 HARDWARE OVERVIEW

The MC68881 and MC68882 are high-performance floating-point devices designed to interface with the MC68020 or MC68030 as coprocessors. These coprocessors fully support the MPU virtual machine architecture and are implemented in HCMOS, Freescale's low-power, small-geometry process. This process allows CMOS and HMOS (high-density NMOS) gates to be combined on the same device. CMOS structures are used where speed and low power are required, and HMOS structures are used where minimum silicon area is desired. As a result, HCMOS technology provides the combined advantages of high-performance and low-power consumption without enlarging the die size.

In systems using the MC68000, MC68008, or MC68010 as the main processor, the MC68881 or MC68882 functions as a peripheral processor. The configuration of the FPCP as a peripheral processor or coprocessor can be completely transparent to user software (i.e., the same object code can be executed in either configuration).

The architecture of the FPCP appears to the user as a logical extension of the M68000 Family architecture. Because of the coupling of the coprocessor interface, the MPU programmer can view the FPCP registers as though the registers were resident in the MPU. Thus, an MPU and FPCP device pair appears to be one processor that supports seven floating-point and integer data types with eight integer data registers, eight address registers, and eight floating-point data registers.

The FPCP programming model is shown in Figures 1-1 through 1-6 and consists of the following:

- Eight 80-bit floating-point data registers (FP7–FP0). These registers are analogous to the integer data registers (D7–D0) and are completely general purpose (i.e., any instruction may use any register).
- A 32-bit control register that contains enable bits for each class of exception trap, and mode bits to set the user-selectable rounding and precision modes.
- A 32-bit status register that contains floating-point condition codes, quotient bits, and exception status information.
- A 32-bit instruction address register that contains the main processor memory address (virtual) of the last floating-point instruction that was executed. This address is used in exception handling to locate the instruction that caused the exception.

The connection between the MPU and the FPCP is a simple extension of the M68000 bus interface. The FPCP is connected as a coprocessor to the MPU, and a chip-select signal (decoded from the MPU function codes and address bus) selects the FPCP. Figure 1-7 illustrates the coprocessor/MPU configuration.

As shown in Figure 1-8, the MC68881 is internally divided into two processing elements; the bus interface unit (BIU) and the arithmetic processing unit (APU). The BIU communicates

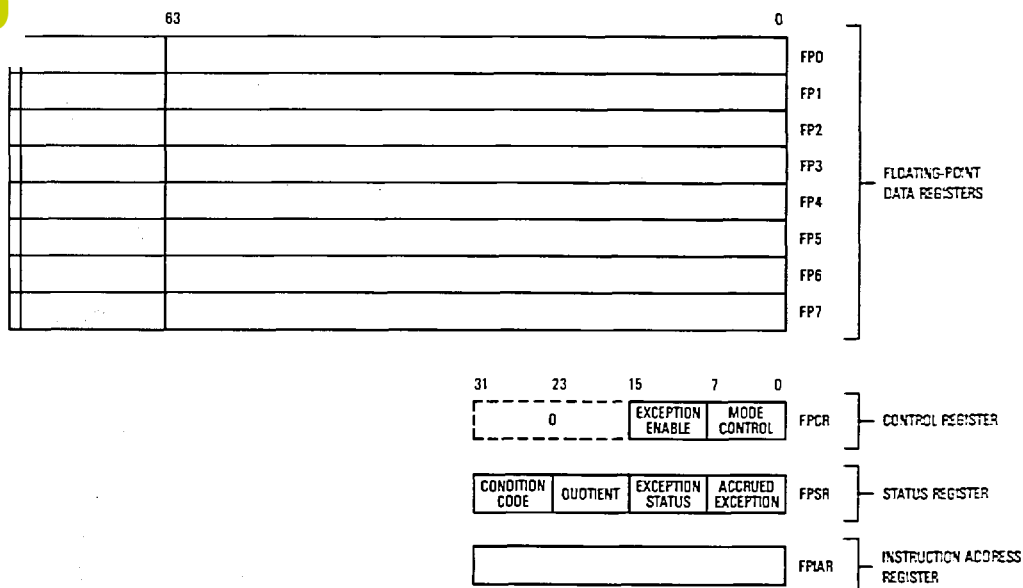


Figure 1-1. MC68881/MC68882 Programming Model

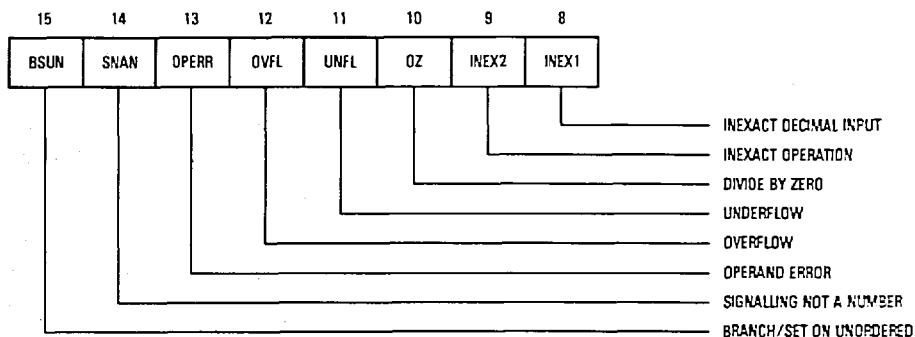


Figure 1-2. Exception Status/Enable Byte

with the MPU, and the APU executes all MC68881 instructions. Though the BIU monitors the state of the APU closely, it operates independently of the APU. The APU operates on the command word and operands that the BIU passes to it. In return, the APU reports its internal status to the BIU.

The BIU contains the coprocessor interface registers (CIRs). In addition to these registers, the CIR register select and $\overline{DSACK}$ timing control logic is contained in the BIU. Finally, the status flags used to monitor the status of communications with the main processor are contained in the BIU.

The eight 80-bit floating-point data registers (FP7–FP0) and the 32-bit control, status, and instruction address registers (FPCR, FPSR, and FPIAR) are located in the APU. In addition

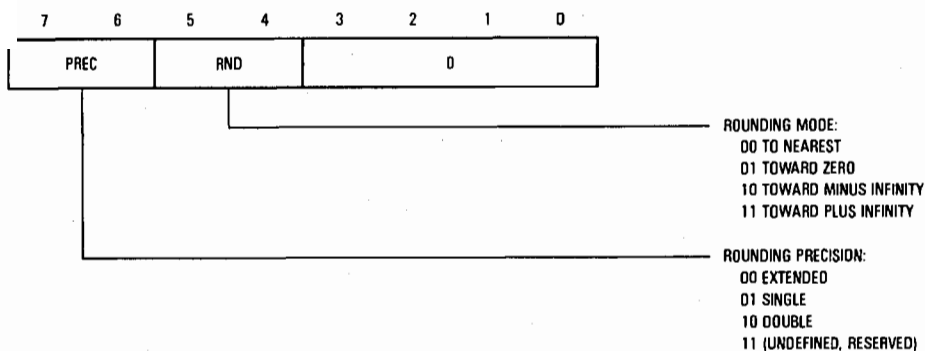


Figure 1-3. Mode Control Byte

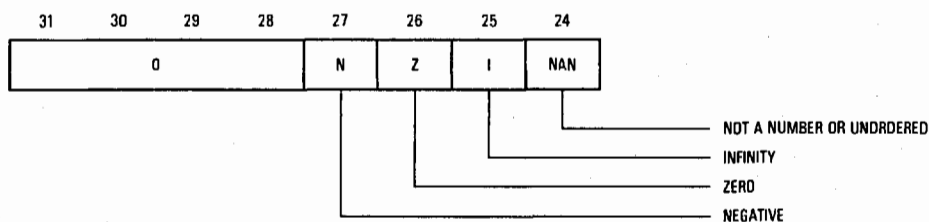


Figure 1-4. Condition Code Byte

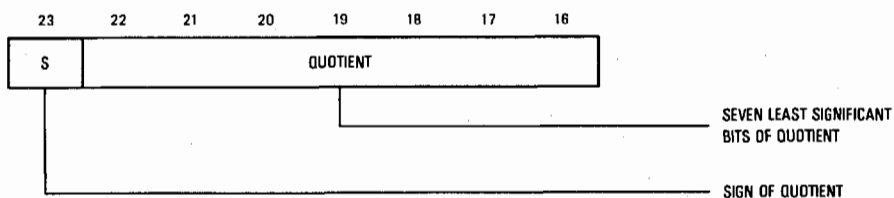


Figure 1-5. Quotient Byte

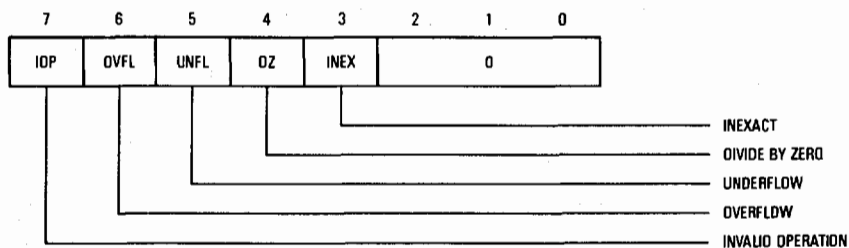


Figure 1-6. Accrued Exception Byte

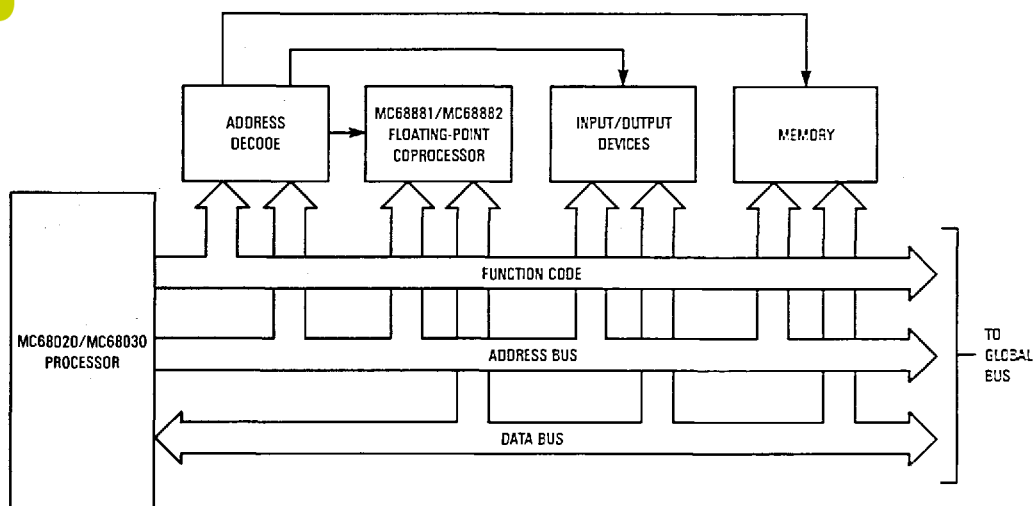


Figure 1-7. Typical Coprocessor Configurations

to these registers, the APU contains a high-speed 67-bit arithmetic unit used for both mantissa and exponent calculations, a barrel shifter that can shift from 1 bit to 67 bits in one machine cycle, and ROM constants (for use by the internal algorithms or user programs).

The control section of the APU contains the clock generator, a two-level microcoded sequencer, the microcode ROM, and self-test circuitry. The built-in self-test capabilities of the FPCP enhance reliability and ease manufacturing requirements; however, these diagnostic functions are not accessible outside of the special test environment supported by VLSI test equipment.

In addition to the BIU and APU, as described for the MC68881 (refer to Figure 1-9), the MC68882 has a conversion unit (CU) that performs data format conversions to the internal extended format. The CU relieves the APU of a significant work load and allows the MC68882 to execute FMOVE instructions concurrently with arithmetic or transcendental operations.

1.2.1 Bus Interface Unit

All communications between the MPU and the FPCP are performed with standard M68000 Family bus transfers. The FPCP is designed to operate on 8-, 16-, or 32-bit data buses.

The FPCP contains a number of coprocessor interface registers (CIRs), which are addressed by the main processor in the same manner as memory. The M68000 Family coprocessor interface is implemented as a protocol of bus cycles in which the main processor reads and writes to these registers. The MPU implements this general-purpose coprocessor interface protocol in hardware and microcode.

When the MPU detects a FPCP general type instruction, the MPU writes the command word of the instruction to the memory-mapped command CIR, and reads the response

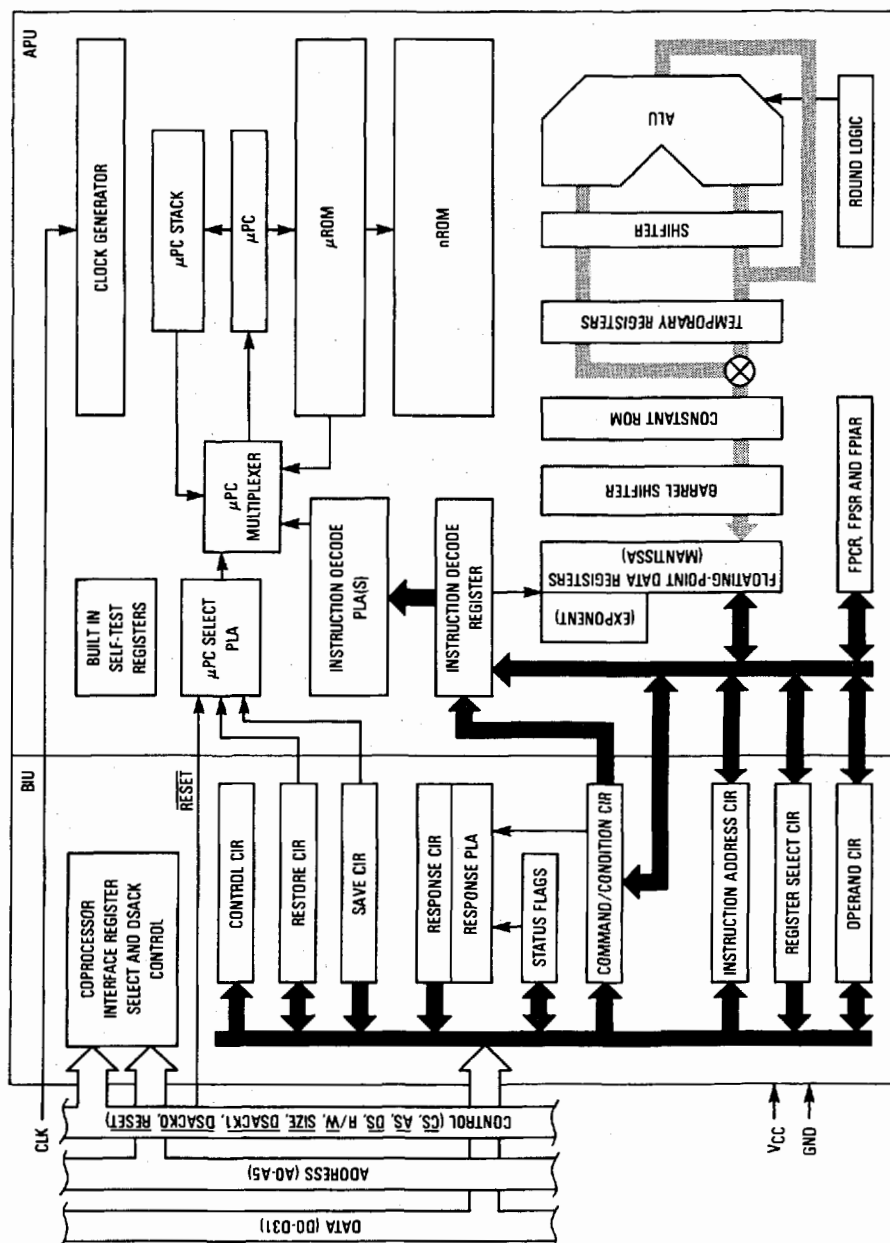


Figure 1-8. MC68881 Simplified Block Diagram

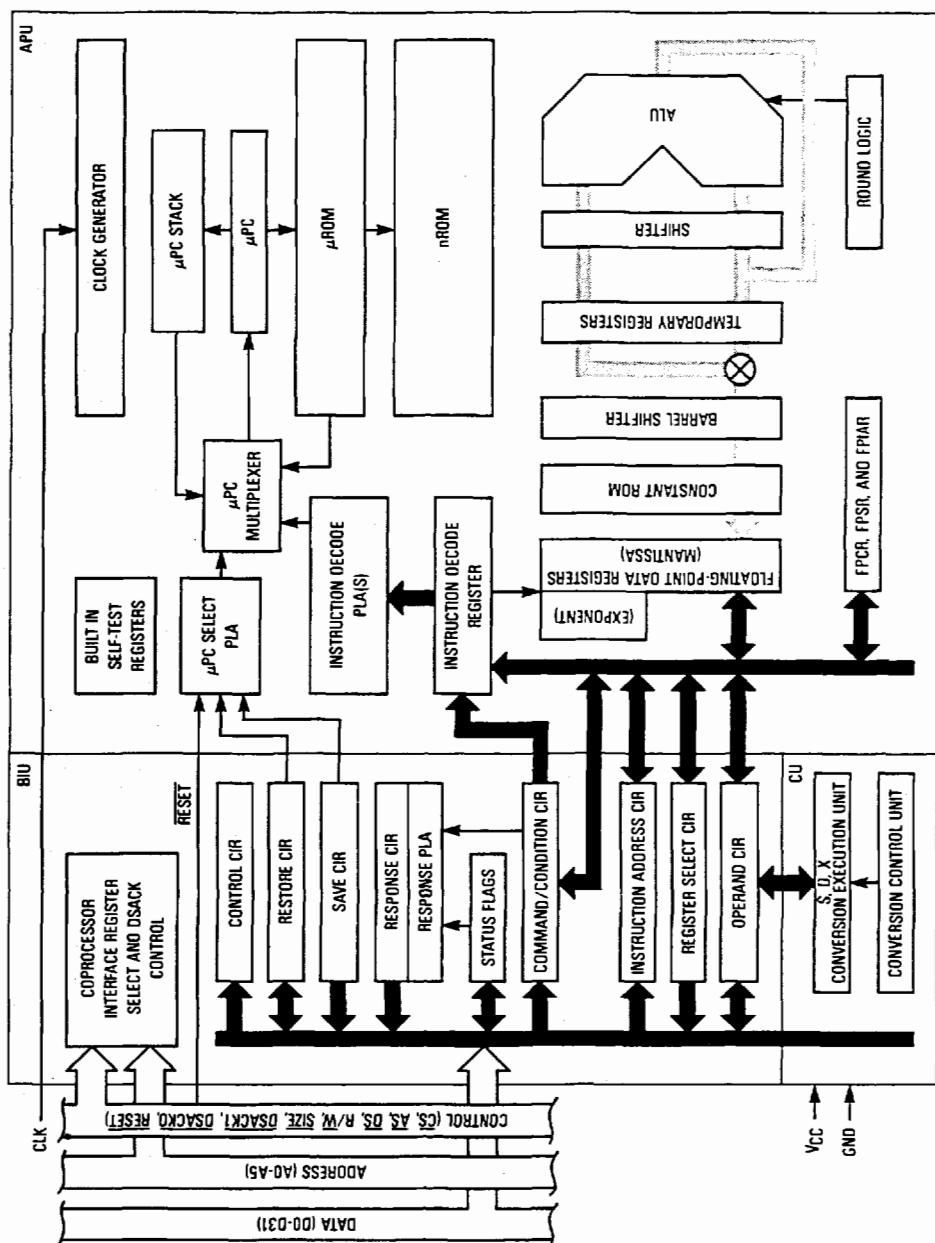


Figure 1-9. MC68882 Simplified Block Diagram

in this response, the BIU encodes requests for any additional service required of the on behalf of the FPCP. For example, the response may request that the MPU fetch an operand from the evaluated effective address and transfer the operand to the operand CIR. Once the MPU fulfills the coprocessor request(s), the MPU is free to fetch and execute subsequent instructions.

A key concern in a coprocessor interface that allows concurrent instruction execution is synchronization during main processor and coprocessor communication. If a subsequent instruction is written to the command CIR before the APU has completed execution of the previous instruction (in the case of the MC68881) or before the CU has passed its results to the APU (in the case of the MC68882), the response instructs the MPU to wait. Thus, the choice of concurrent or nonconcurrent instruction execution is determined on an instruction-by-instruction basis by the coprocessor.

The only difference between a coprocessor bus transfer and any other bus transfer by the MPU is that the function code issued by the MPU specifies the CPU address space during the cycle (the function codes are generated by the M68000 Family processors to identify one of eight separate address spaces). Thus, the memory-mapped coprocessor interface registers do not infringe upon instruction or data address spaces. The MPU places a coprocessor ID field from the coprocessor instruction words onto three of the upper address lines during coprocessor accesses. This ID, along with the CPU address space function code, is decoded to select one of eight possible coprocessors in the system.

Since the coprocessor interface protocol consists solely of bus transfers, the protocol is easily emulated by software when the FPCP is used as a peripheral with any processor capable of memory-mapped I/O over an M68000-style bus. When used as a peripheral processor with the 8-bit MC68008 or either the 16-bit MC68000 or MC68010, all FPCP instructions are trapped by the main processor to an exception handler at execution time. Trapping the instructions enables the software emulation of the coprocessor interface protocol to be totally transparent to the user. The FPCP can provide a performance option for MC68000-based designs by changing the main processor to an MC68020 or MC68030. The software migrates without change to the next generation equipment using the MPU.

Since the bus is asynchronous, the FPCP need not run at the same clock speed as the main processor. Total system performance can therefore be customized. For a given CPU performance requirement, the floating-point performance can be selected to meet particular price/performance specifications, running the FPCP at slower (or faster) clock speeds than the CPU clock.

1.2.2 Coprocessor Interface

The M68000 Family coprocessor interface is an integral part of the FPCP and MPU designs. The interface partitions MPU and coprocessor operations so that the MPU does not have to completely decode coprocessor instructions, and the FPCP does not have to duplicate main processor functions (such as effective address evaluation).

This partitioning provides an orthogonal extension of the instruction set by permitting FPCP instructions to utilize all MPU addressing modes and to generate execution time exception traps. Thus, from the programmer's view, the MPU and coprocessor appear to be integrated onto a single chip. While the execution of the great majority of FPCP instructions can be overlapped with the execution of MPU instructions, concurrency is completely



parent to the programmer. The MPU single-step and program flow (trace) modes are supported by the FPCP and the M68000 Family coprocessor interface.

While the M68000 Family coprocessor interface permits coprocessors to be bus masters, the FPCP never functions as one. The FPCP requests that the MPU fetch all operands and store all results. In this manner, the MPU 32-bit data bus provides high-speed transfer of floating-point operands and results while reducing the pin count of the FPCP.

Since the coprocessor interface consists solely of bus cycles (to and from the CPU space) and the FPCP never functions as a bus master, the coprocessor can be placed on either the logical or physical side of the system memory management unit (MMU) in an MC68020 system. Since the MMU of the MC68030 is on-chip, the FPCP is always on the physical side of the MMU in an MC68030 system.

The virtual machine architecture of the MPU is supported by the coprocessor interface and the FPCP with the FSAVE and FRESTORE instructions. If the MPU detects a page fault and/or a task timeout, the MPU can force the FPCP to stop whatever operation is in progress at any time and save the FPCP internal state in memory. During the execution of a floating-point instruction, the FPCP can stop at predetermined points as well as at the completion of the instruction.

The size of the saved internal state of the FPCP is dependent upon the state of the APU at the time that the FSAVE is executed. If the MPU is in the reset state when the FSAVE instruction is initiated, only one word of state is transferred to memory. The stored word may be examined by the operating system to determine that the coprocessor programmer's model is empty. If the APU is in the idle state when the FSAVE instruction is decoded, only a few words of internal state are transferred to memory. If the APU is in the middle of executing an instruction, it may be necessary to save the entire internal state of the machine. Instructions that can complete execution in less time than it would take to save the larger state in mid-instruction are automatically allowed to complete execution and then save the idle state. Thus, the size of the saved internal state is kept to a minimum. The ability to utilize several internal state sizes greatly reduces the average context switching time.

The FRESTORE instruction permits reloading an internal state saved earlier and continues any previously suspended operation. Restoring the reset internal state re-establishes default register values, a function identical to the FPCP hardware reset.

1.3 OPERAND DATA FORMATS

The FPCP supports the following data formats:

- Byte Integer (B)
- Word Integer (W)
- Long Word Integer (L)
- Single Precision Real (S)
- Double Precision Real (D)
- Extended Precision Real (X)
- Packed Decimal String Real (P)

The capital letters within the parentheses denote suffixes added to mnemonics of the assembly language instructions to specify the data format to be used.

The three integer data formats (byte, word, and long word) are the standard two's complement data formats defined in the M68000 Family architecture. Whenever an integer is used in a floating-point operation, the integer is automatically converted by the FPCP to an extended precision floating-point number before being used. For example, to add an integer constant of five to the number in floating-point data register 3 (FP3), the following instruction can be used:

```
FADD.W #5,FP3
```

(The Freescale assembler syntax uses “#” to denote immediate addressing.)

The ability to effectively use integers in floating-point operations saves user memory since an integer representation of a number, if representable, is usually smaller than the equivalent floating-point representation.

1.3.2 Floating-Point Data Formats

The floating-point data formats, single precision (32-bits) and double precision (64-bits), are implemented in the FPCP as defined by the IEEE standard. These data formats are the main floating-point formats and should be used for most calculations involving real numbers. Table 1-1 lists the exponent and mantissa sizes for single, double, and extended precision. The exponent is biased, and the mantissa is in sign and magnitude form. Since single and double precision require normalized numbers, the most significant bit of the mantissa is implied as a one and is not included, thus giving one extra bit of precision.

Table 1-1. Exponent and Mantissa Sizes

Data Format	Exponent Bits	Mantissa Bits
Single	8	23(+ 1)
Double	11	52(+ 1)
Extended	15	64

The extended precision data format is also in conformance with the IEEE standard, but the standard does not specify this format to the bit level as it does for single and double precision. The memory format for the FPCP consists of 96 bits (three long words). Only 80 bits are actually used; the other 16 bits are for future expandability and for long-word alignment of floating-point data structures in memory. Extended format has a 15-bit exponent, a 64-bit mantissa, and a 1-bit mantissa sign.

Extended precision numbers are intended for use as temporary variables, intermediate values, or where extra precision is needed. For example, a compiler might select extended precision arithmetic for evaluation of the right side of an equation with mixed sized data and then convert the answer to the data type on the left side of the equation. It is anticipated that extended precision numbers will not be stored in large arrays due to the amount of memory required by each value.

1.3.3 Packed Decimal String Real Data Format

The packed decimal data format allows packed BCD strings to be transferred to and from the FPCP. The strings consist of a 3-digit base 10 exponent and a 17-digit base 10 mantissa.



the exponent and mantissa have separate sign bits. All digits are packed BCD, and the string fits in 96 bits (three long words). As is the case with all data formats, when packed BCD strings are supplied to the FPCP, the strings are automatically converted to extended precision real values. This conversion allows packed BCD numbers to be used as inputs to any operation. For example:

```
FADD.P  # -6.023E + 24,FP5
```

BCD numbers can be supplied by the FPCP in a format readily used for printing by a program generated by a high-level language compiler. For example:

```
FMOVE.P  FP3,BUFFER{# - 5}
```

This instruction converts the contents of floating-point data register 3 (FP3) into a packed BCD string with five significant digits to the right of the decimal point (FORTRAN F format).

1.3.4 Data Format Summary

All data formats described in the preceding paragraphs are supported orthogonally by all arithmetic and transcendental operations and by all appropriate MPU addressing modes. For example, all of the following are valid instructions:

```
FADD.B    #0,FP0
FADD.W    D2,FP3
FADD.L    BIGINT,FP7
FADD.S    #3.14159,FP5
FADD.D    (SP)+,FP6
FADD.X    [(TEMP_PTR,A7)],FP3
FADD.P    #1.23E25,FP0
```

Most on-chip calculations are performed in the extended precision format, and the eight floating-point data registers always contain extended precision values. All operands are converted to extended precision by the FPCP before a specific operation is performed, and all results are in extended precision. This ensures maximum accuracy without sacrificing performance.

Refer to Figure 1-10 for a summary of the memory formats for the seven data formats supported by the FPCP.

1.4 INSTRUCTION SET

The FPCP instruction set is organized into six major classes:

1. Moves between the FPCP and memory or the MPU (to or from)
2. Move multiple registers (to or from)
3. Monadic operations
4. Dyadic operations
5. Branch, set, or trap conditionally
6. Miscellaneous

1.4.1 Moves

On all moves from memory (or from an MPU data register) to the FPCP, data is converted from the source data format to the internal extended precision format. On all moves from

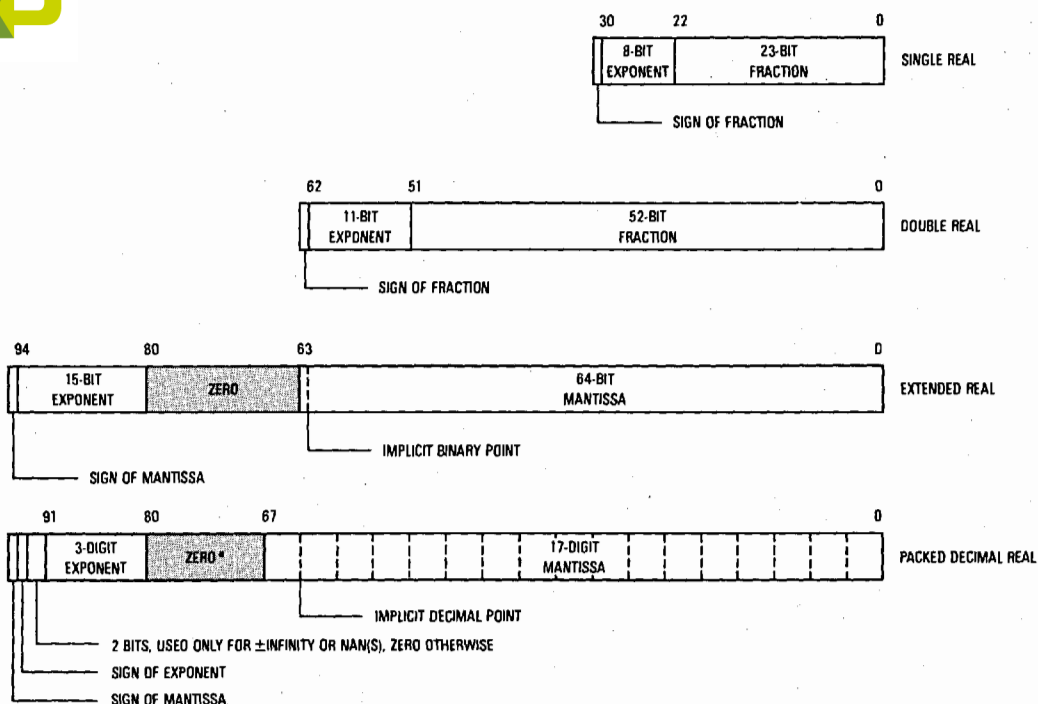


Figure 1-10. MC68881/MC68882 Data Format Summary

the FPCP to memory (or to an MPU data register), data is converted from the internal extended precision format to the destination data format.

Note that data movement instructions perform arithmetic operations, since the result is always rounded to the precision selected in the FPCR mode control byte. The result is rounded using the selected rounding mode and is checked for overflow and underflow.

The syntax for the FMOVE instruction is:

```
FMOVE.<fmt> <ea>,FPn   Move to FPCP
FMOVE.<fmt>  FPm,<ea>   Move from FPCP
FMOVE.X      FPm,FPn    Move within FPCP
```

where:

- <ea> is an MPU effective address operand
- <fmt> is the data format size
- FPm and FPn are floating-point data registers.

1.4.2 Move Multiple Registers

The floating-point move multiple instruction on the FPCP resembles its integer counterpart on the M68000 Family processors. Any set of the floating-point registers FP0 through FP7



are moved to or from memory with one instruction. These registers are always moved bit extended data with no conversion (hence no possibility of conversion errors).

Some examples of the move multiple instruction are as follows:

```
FMOVEM    <ea>,FP0-FP3/FP7
FMOVEM    FP2/FP4/FP6,<ea>
```

The move multiple instruction is useful during context switches and interrupts to save or restore the state of a program. It is also useful at the start and end of a procedure to save and restore the calling routine's register set. In order to reduce procedure call overhead, the list of registers to be saved or restored can be stored in a data register thus enabling run-time optimization by allowing a called routine to save as few registers as possible. Note that no rounding or overflow/underflow checking is performed by these operations.

1.4.3 Monadic Operations

A monadic operation has one operand. This operand may be in a floating-point data register, in memory, or in an MPU data register. The result is always stored in a floating-point data register. For example, the syntax for square root is any of the following:

```
FSQRT.<fmt>    <ea>,FPn
FSQRT.X        Fpm,FPn
FSQRT.X        FPn
```

The monadic operations available with the FPCP are as follows:

FABS	Absolute Value	FLOG2	Log Base 2
FACOS	Arc Cosine	FLOGN	Log Base e
FASIN	Arc Sine	FLOGNP1	Log Base e of (x + 1)
FATAN	Arc Tangent	FNEG	Negate
FATANH	Hyperbolic Arc Tangent	FSIN	Sine
FCOS	Cosine	FSINCOS	Simultaneous Sine and Cosine
FCOSH	Hyperbolic Cosine	FSINH	Hyperbolic Sine
FETOX	e to the x Power	FSQRT	Square Root
FETOXM1	e to the x Power - 1	FTAN	Tangent
FGETEXP	Get Exponent	FTANH	Hyperbolic Tangent
FGETMAN	Get Mantissa	FTENTOX	10 to the x Power
FINT	Integer Part	FTST	Test
FINTRZ	Integer Part (Truncated)	FTWOTOX	2 to the x Power
FLOG10	Log Base 10		

1.4.4 Dyadic Operations

Dyadic operations have two operands each. The first operand is in a floating-point data register, memory, or an MPU data register. The second operand is the contents of a floating-point data register. The destination is the same floating-point data register used for the second operand. For example, the syntax for floating-point add is either of the following:

```
FADD.<fmt>    <ea>,FPn
FADD.X        Fpm,FPn
```

The dyadic operations available with the FPCP are as follows:

FADD	Add	FREM	IEEE Remainder
FCMP	Compare	FSCALE	Scale Exponent

Assuming that operands are single precision, the FSGLMUL and FSGLDIV instructions round results as such while maintaining the range of extended precision. In special applications where multiply and divide performance are more important than loss of precision, the FSGLMUL and FSGLDIV instructions can be used.

1.4.5 Branch, Set, and Trap-On Condition

The floating-point branch, set, and trap-on condition instructions implemented by the FPCP are similar to the equivalent integer instructions of the M68000 Family processors, except more conditions exist due to the special values in IEEE floating-point arithmetic. When a conditional instruction is executed, the FPCP performs the necessary condition checking and reports the result, true or false, to the MPU; the MPU then takes the appropriate action. Since the FPCP and MPU are closely coupled, the floating-point branch operations are quickly executed.

The FPCP conditional operations are:

FBcc	Branch
FDBcc	Decrement and Branch
FScc	Set According to Condition
FTRAPcc	Trap-on Condition (with an Optional Parameter)

where:

cc is one of the 32 floating-point conditional test specifiers listed in **3.3 PACKED DECIMAL REAL DATA FORMAT**.

1.4.6 Miscellaneous Instructions

Miscellaneous instructions include moves to and from the status, control, and instruction address registers. Also included are the virtual memory/machine FSAVE and FRESTORE instructions that save and restore the internal state of the FPCP.

FMOVE	<ea>,FPcr	Move to Control Register(s)
FMOVE	FPcr,<ea>	Move from Control Register(s)
FSAVE	<ea>	Virtual Machine State Save
FRESTORE	<ea>	Virtual Machine State Restore

1.5 ADDRESSING MODES

The FPCP does not perform address calculations. Thus, when the FPCP instructs the MPU to transfer an operand via the coprocessor interface, the MPU performs the addressing mode calculations requested in the instruction. In this case, the instruction is encoded specifically for the MPU, and the instruction execution by the FPCP is dependent only on the value of the command word written to the FPCP by the main processor.

This interface is quite flexible and allows any addressing mode to be used with floating-point instructions. For the M68000 Family, these addressing modes include immediate, postincrement, predecrement, data or address register direct, and the indexed/indirect



ssing modes of the MPU. Some addressing modes are restricted for some instructions ping with the M68000 Family architectural definitions (e.g., program counter relative addressing is not allowed for a destination operand).

The orthogonal instruction set of the FPCP, along with the flexible branches and addressing modes of the MPU, allows a programmer or a compiler writer to think of the FPCP as though it were part of the MPU. There are no special restrictions imposed by the coprocessor interface, and floating-point arithmetic is coded exactly like integer arithmetic.

1.6 MC68882 PROGRAMMING CONSIDERATIONS

To exploit the enhanced performance of the MC68882 requires the programmer to be aware of the manner in which the coprocessor overlaps execution of instructions. Upgrading a system to use the MC68882 requires minor system software changes but no user software changes. To optimize applications code for the MC68882 may require reordering of floating-point instructions. **SECTION 5 COPROCESSOR PROGRAMMING** describes the concurrency capabilities of the MC68882, the required system software changes, and the optimization of existing software for the enhanced floating-point coprocessor.